

Figure 1: Rumoca



I did not have time to figure out how to hook up the generated code with a IVP solver. It seems like the SUNDIALS4py IVA solver would be ideal since Rumoca uses sundials to solve DAE equations. The trajectory here is more accurate than what I was using earlier because for my python code I was only sampling the twist every 0.1 seconds so it did not flip the angular velocity direction exactly at the right time, leading to an offset lower lobe. The Modelica trajectory is better because it solves the underlying differential equation instead of discretizing it.

Code Snippet 1: Rumoca CodeGen

```

1
2 """
3 Generated by Rumoca.
4
5 This template renders the current DAE context into a SymPy-friendly symbolic
6 model. It keeps Rumoca's continuous equations in residual form and can solve
7 for explicit state derivatives and algebraics when possible.
8 """
9
10 import sympy as sp
11
12
13 def der(symbol):
14     if isinstance(symbol, sp.Indexed):
15         return sp.IndexedBase(f"{symbol.base}_dot")[symbol.indices]
16     if isinstance(symbol, sp.IndexedBase):
17         return sp.IndexedBase(f"{symbol}_dot")
18     return sp.Symbol(f"{symbol}_dot")
19
20
21 def pre(symbol):
22     if isinstance(symbol, sp.Indexed):
23         return sp.IndexedBase(f"pre_{symbol.base}")[symbol.indices]
24     if isinstance(symbol, sp.IndexedBase):
25         return sp.IndexedBase(f"pre_{symbol}")
26     return sp.Symbol(f"pre_{symbol}")
27
28
29 def if_else(condition, then_expr, else_expr):
30     return sp.Piecewise((then_expr, condition), (else_expr, True))
31
32
33 def _vector(entries):
34     return (
35         sp.Matrix([symbol for _, symbol, _ in entries]),
36         {name: start for name, _, start in entries},
37         {name: index for index, (name, _, _) in enumerate(entries)},
38     )
39
40
41 def _column(entries):

```

```

42     return sp.Matrix(entries) if entries else sp.Matrix([])
43
44
45 def zeros(*dims):
46     if len(dims) == 1:
47         return sp.Matrix.zeros(int(dims[0]), 1)
48     return sp.Matrix.zeros(*(int(dim) for dim in dims))
49
50
51 def ones(*dims):
52     if len(dims) == 1:
53         return sp.Matrix.ones(int(dims[0]), 1)
54     return sp.Matrix.ones(*(int(dim) for dim in dims))
55
56
57 def sum(values):
58     if isinstance(values, sp.MatrixBase):
59         return sp.Add(*list(values))
60     if isinstance(values, (list, tuple)):
61         return sp.Add(*values)
62     return values
63
64
65 abs = sp.Abs
66 min = sp.Min
67 max = sp.Max
68 sign = sp.sign
69 sqrt = sp.sqrt
70 sin = sp.sin
71 cos = sp.cos
72 tan = sp.tan
73 asin = sp.asin
74 acos = sp.acos
75 atan = sp.atan
76 atan2 = sp.atan2
77 sinh = sp.sinh
78 cosh = sp.cosh
79 tanh = sp.tanh
80 exp = sp.exp
81 log = sp.log
82 log10 = lambda value: sp.log(value, 10)
83 floor = sp.floor
84 ceil = sp.ceiling
85 transpose = lambda value: sp.Matrix(value).T
86 identity = sp.eye
87 cross = lambda lhs, rhs: sp.Matrix(lhs).cross(sp.Matrix(rhs))
88 div = lambda lhs, rhs: sp.floor(lhs / rhs)
89 mod = sp.Mod
90 rem = sp.Mod
91
92
93 class Model:
94     """Flattened symbolic DAE model."""
95
96     def __init__(self):
97         self.time = sp.Symbol("time")
98         time = self.time
99
100         # States, algebraics, inputs, outputs, parameters, constants, discreties.
101
102
103         theta = sp.Symbol("theta")
104
105
106         x = sp.Symbol("x")
107
108
109         y = sp.Symbol("y")

```

```

110
111
112     self.x, self.x_start, self.x_index = _vector([
113         ("theta", theta, 0),
114         ("x", x, 0),
115         ("y", y, 0)
116     ])
117
118
119
120     omega_adj = sp.Symbol("omega_adj")
121
122
123     tau = sp.Symbol("tau")
124
125
126     self.y, self.y_start, self.y_index = _vector([
127         ("omega_adj", omega_adj, 0),
128         ("tau", tau, 0)
129     ])
130
131
132     self.u, self.u_start, self.u_index = _vector([])
133
134
135     self.w, self.w_start, self.w_index = _vector([])
136
137
138
139     T = sp.Symbol("T")
140
141
142     T_fig8 = sp.Symbol("T_fig8")
143
144
145     T_lobe = sp.Symbol("T_lobe")
146
147
148     omega_turn = sp.Symbol("omega_turn")
149
150
151     v_nom = sp.Symbol("v_nom")
152
153
154     self.p, self.p_start, self.p_index = _vector([
155         ("T", T, 35),
156         ("T_fig8", T_fig8, (10 * pi)),
157         ("T_lobe", T_lobe, (5 * pi)),
158         ("omega_turn", omega_turn, 0.4),
159         ("v_nom", v_nom, 2)
160     ])
161
162
163
164     pi = sp.Symbol("pi")
165
166
167     self.constants, self.constants_start, self.constants_index = _vector([
168         ("pi", pi, 3.141592653589793)
169     ])
170
171
172     self.z, self.z_start, self.z_index = _vector([])
173
174
175     self.m, self.m_start, self.m_index = _vector([])
176
177

```

```

178     self.x_dot_alias, self.x_dot_alias_start, self.x_dot_alias_index = _vector([])
179
180     self.f_x = _column([
181         (der(x) - (cos(theta) * v_nom)),
182         (der(y) - (sin(theta) * v_nom)),
183         (tau - (time - (T_fig8 * floor((time / T_fig8))))),
184         (omega_adj - if_else((tau > T_lobe), (-omega_turn), omega_turn)),
185         (der(theta) - omega_adj)
186     ])
187
188     self.initial_residuals = _column([])
189
190     self.relations = _column([
191         (tau > T_lobe)
192     ])
193
194     self.explicit_targets = [
195
196         der(x),
197
198
199         der(y),
200
201
202         tau,
203
204
205         omega_adj,
206
207
208         der(theta),
209
210     ]
211     self.explicit_solution = None
212     self.explicit_rhs = sp.Matrix([])
213
214     def solve_explicit(self):
215         """Best-effort symbolic solve for explicit derivatives and algebraics."""
216         if len(self.f_x) == 0 or len(self.explicit_targets) == 0:
217             self.explicit_solution = {}
218             self.explicit_rhs = sp.Matrix([])
219             return self.explicit_solution
220
221         solutions = sp.solve(
222             list(self.f_x),
223             self.explicit_targets,
224             dict=True,
225             simplify=False,
226         )
227         self.explicit_solution = solutions[0] if solutions else {}
228         self.explicit_rhs = sp.Matrix(
229             [
230                 self.explicit_solution.get(symbol, symbol)
231                 for symbol in self.explicit_targets
232             ]
233         )
234         return self.explicit_solution
235
236     def summary(self):
237         return {
238             "states": [str(symbol) for symbol in self.x],
239             "algebraics": [str(symbol) for symbol in self.y],
240             "inputs": [str(symbol) for symbol in self.u],
241             "parameters": [str(symbol) for symbol in self.p],
242             "continuous_residual_count": len(self.f_x),
243             "initial_residual_count": len(self.initial_residuals),
244             "explicit_target_count": len(self.explicit_targets),
245         }

```

```
246
247
248 if __name__ == "__main__":
249     model = Model()
250     print(model.summary())
251     if len(model.f_x) > 0 and len(model.explicit_targets) > 0:
252         print(model.solve_explicit())
```